SpellSprint Product Requirements Document

https://gemini.google.com/app/dd139e07b6bf777b

User prompt: "A free, browser-based multiplayer spelling game. Players join a room via a shared link (no install). Each round shows a picture of a common thing (e.g. a pomegranate, broccoli, a giraffe) and players race to type its correct spelling. Points for correct and fast answers." So, this is what I want to build. Let's get into it. How we are going to build it. Let's create a PRD and then let's discuss the things. Basically, the product part first, and then I will be building this on top of Cloudflare. I have a starter project, I will attach the link to the starter project, and then we can discuss the back-end part first, the data models, etc. Then we can discuss the API part. We can build that first. I don't care if the player doesn't have any account, he can just join in as something and then create an thing, you know, just we give them a default name and then he can continue to play if he wants, he can create an account. If not, it's also fine, I guess we will allow him to create a name if he wants, then he can either play solo or join a group so join a room etc would also be there then we'll have to think about the architecture of how that will look like but let's complete the product first once he joins the room, it'll start. Let's discuss you. Go ahead and give me a rough sketch. What would you suggest? And let's start thinking about the edge cases as well.

Response: Product Requirements Document (PRD): SpellSprint

1. Executive Summary & Vision

SpellSprint is a zero-friction, browser-based multiplayer spelling race. Players join via a shareable URL with no account creation or app installation required. Each round displays a curated photo of a real-world object (e.g., *pomegranate*, *broccoli*, *accordion*), and players race to submit the correct spelling. Points are awarded based on correctness, speed, and placement.

The product goal is casual party play: sub-second onboarding, low barrier to entry, responsive real-time interaction, and minimal infrastructure overhead.

2. User Personas & Core Journeys

Persona	Motivation	Primary Flow
The Room Host	Wants to play a 10-minute game with friends over Discord, Slack, or in-person.	Lands on site → Clicks "Create Private Room" → Copies invite link → Starts game when friends join.
The Guest Player	Received a link on mobile or desktop; wants to play immediately without signups.	Clicks link → Lands in lobby with a random default name (`SwiftFox84`) → Hits "Ready" or edits name → Plays.
The Solo Sprinter	Wants a quick mental workout or spelling drill during downtime.	Lands on site → Clicks "Solo Blitz" → Plays timed rounds against a personal high score.

3. Core Mechanics & Game Loop

```
[ Lobby / Invite ]
       │
       ▼
[ 3-Second Countdown ] ──► Pre-fetches round image assets
       │
       ▼
[ Active Round (15s) ] ──► Image shown, letter-count blanks displayed
       │                   Players submit spellings via input field
       │                   Server validates each guess in real-time
       ▼
[ Round Reveal (5s) ]  ──► Target word revealed, pronunciation audio plays
       │                   Round scores & speed rankings updated
       ▼
(Repeat for N rounds)
       │
       ▼
[ Match Podium ]       ──► Final 1st/2nd/3rd places, summary stats, Rematch button
```

Guess Validation & Hints

1. **Input Rules:** Submissions are case-insensitive and trim leading/trailing whitespace.
2. **Partial Feedback:** If a guess is incorrect, the input field provides a rapid shake animation. If the guess is within a Levenshtein distance of 1 (e.g., *pomegrante* instead of *pomegranate*), a transient toast ("Almost! Check your spelling")

appears without burning an attempt.

3. **Lock-In:** Once a player submits the correct word, their input locks for that round. Their status changes to "Solved" on the live roster, concealing the answer from other players who are still guessing.
4. **Early Termination:** If all active players solve the word before the timer expires, the round terminates immediately and skips to the reveal phase.

Scoring Formula

Points reward both speed and accuracy. Each round provides a maximum of 1,000 base points:

$$\text{Points} = \text{Base Correctness } (500) + \left(\frac{t_{\text{remaining}}}{t_{\text{total}}} \times 400 \right) + \text{First Solver Bonus } (100)$$

- **First Solver Bonus:** Flat $+100$ awarded exclusively to the fastest correct answer in that round.
- **Streak Multiplier:** Consecutive correct rounds apply a $+10\%$ bonus per streak level (capped at $+50\%$).

4. Functional Requirements

4.1 Identity & Session Management

- **P0 - Anonymous Default Identity:** Every visitor receives a persistent client UUID (stored in `localStorage`) paired with an auto-generated guest handle (e.g., `CleverOtter31`) and random geometric avatar.
- **P0 - Inline Name Editing:** Users can edit their display name at any point in the lobby before a game starts. Names are sanitized (HTML stripped, 15-character limit, client/server profanity block).
- **P2 - Account Claiming:** Users can optionally link an email or Google account to persist match history, global leaderboard ranking, and unlocked avatars across devices.

4.2 Room & Lobby System

- **P0 - One-Click Room Creation:** Generates a 6-character alphanumeric room code and short URL (e.g., `spellsprint.io/r/K9XZ2A`).
- **P0 - Room Presence & State Sync:** WebSockets broadcast lobby member join/leave events, ready states, and room configuration changes.
- **P0 - Configurable Room Settings:**
  - Number of rounds: 5, 10, 15 (Default: 10).
  - Round time limit: 10s, 15s, 20s (Default: 15s).
  - Word difficulty: Easy, Medium, Hard, Mixed (Default: Mixed).
- **P1 - Host Controls:** The room creator can kick disruptive players, transfer host privileges, or force-start the match.

4.3 Gameplay Engine

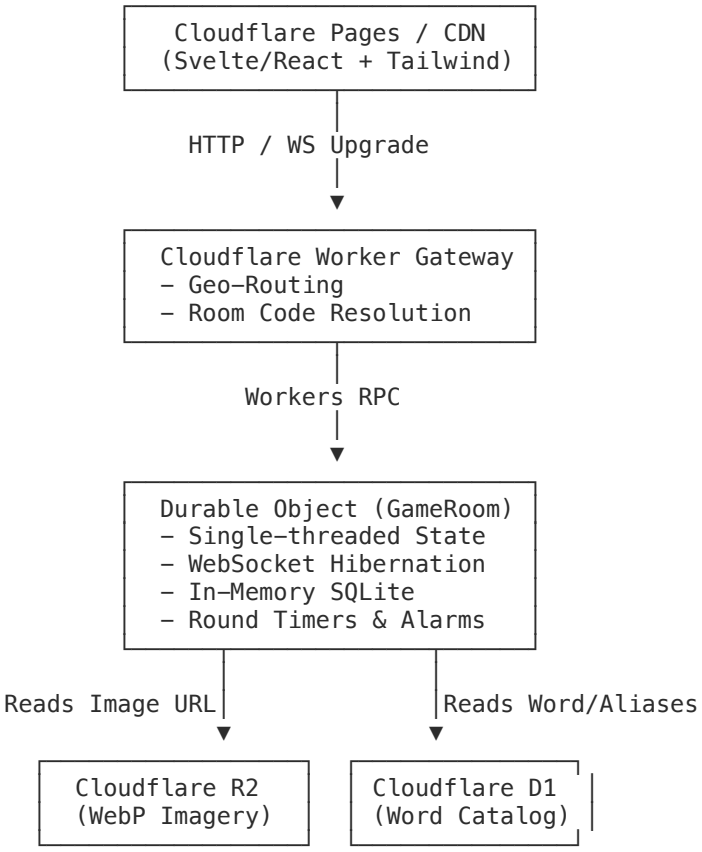
- **P0 - Server-Authoritative State:** The client never knows the correct word in advance. The server sends only the `roundId`, `imageUrl`, and `characterCount` (e.g., 11 letters for *pomegranate*).
- **P0 - Guess Evaluation:** Every guess is sent over the WebSocket. The server compares the normalized string against canonical names and accepted aliases.
- **P0 - Live Broadcast of Progress:** When Player A solves the word, all other connected clients receive an event: `{ type: "PLAYER_SOLVED", playerId: "p1", place: 1 }`. The word itself is not broadcast.
- **P0 - Match End Summary:** Displays final podium, total points, average solve time, and hardest word of the match (word with lowest solve percentage).

5. Edge Cases & Handling Strategy

Scenario	Risk	Mitigation
Client Inspects Network / DOM	Cheating by reading the target answer from JavaScript variables or network payloads.	**Zero-Knowledge Client:** Target words are never transmitted to the browser during an active round. Guess evaluation is strictly executed server-side.

Scenario	Risk	Mitigation
Synonyms & Regional Dialects	A photo of an *eggplant* is submitted as *aubergine*, or *doughnut* vs. *donut*.	**Alias Mapping:** The data model supports an array of acceptable answers (`canonical: "eggplant"`, `aliases: ["aubergine", "brinjal"]`). Any match awards full points.
Host Disconnects Mid-Match	Game hangs indefinitely waiting for host actions.	**Host Migration:** If the host drops for > 5 seconds, host privileges automatically transfer to the player with the earliest connection timestamp.
Player Reconnects Mid-Round	Page refresh or mobile network handoff causes loss of game state and score.	**Session Token Recovery:** Client retains a signed session token. Reconnecting sends a `RESUME_SESSION` frame, restoring current score, streak, and lock-in state without re-creating the player slot.
Clock Drift / Latency Advantage	Client clocks differ; a player with a delayed clock claims more remaining time.	**Server-Enforced Tick:** Round start and end timestamps are determined solely by the server. Speed points are calculated from `server_received_at − server_round_start`.
Late Joiners	A player joins via URL while Round 4 is currently in progress.	**Spectator Mode:** Late joiners enter in `SPECTATING` state with synchronized live views. They can actively chat or join as a competitor starting the next full match.
Abandoned / Zombie Rooms	Abandoned WebSocket connections consume compute resources.	**Inactivity Teardown:** If a room has zero active connections or zero input for 10 minutes, the room state is garbage-collected.

6. High-Level Technical Architecture (Cloudflare Alignment)



- **Cloudflare Workers & Pages:** Serves the frontend bundle and handles initial HTTP routing.
- **Durable Objects (DO):** Acts as the dedicated room coordinator. Because each DO instance runs in a single-threaded environment with collocated storage and WebSocket Hibernation support, race conditions during simultaneous guess submissions are eliminated without requiring distributed locks.
- **Cloudflare R2:** Stores compressed, optimized WebP imagery served via Cloudflare's edge cache to keep image load times under 100ms.
- **Cloudflare D1:** Houses the curated catalog of objects, target spellings, aliases, and difficulty tiers.

7. Word Catalog Schema Specification

Every round references an asset conforming to this structure:

```
{
  "id": "wrd_8f93a102",
  "canonical_spelling": "pomegranate",
  "acceptable_aliases": [],
  "character_count": 11,
  "difficulty": "hard",
  "category": "fruits_and_food",
  "image_url": "https://assets.spellsprint.io/items/pomegranate.webp",
  "hint_mask": "_ _ _ _ _ _ _ _ _ _ _"
}
```

What starter project template or stack (e.g., React, SvelteKit, Hono, standard Cloudflare Worker templates) are you planning to share so we can tailor the Durable Object state machine and WebSocket protocol directly to it?

User prompt: 1. no need pronunciation audio 2. we give hints for hard words, 1-2 letters only 3. should be mobile first 4. base prefer to be in 100s not thousands, even less is better. no streak multiplier. no first solver bonus 5. Number of rounds: 5. 6. no host controls, should not be able to kick ppl, if they leave host privileges transfer to someone else 7. Match and summary should only have display, final podium, and points of each person and average all time. 8. I don't think we can do synonyms because I was thinking to show the number of spaces, which means the number of character count as space when they type, each one fills in that space. We can discuss more on this though. 9. no chat feat https://github.com/claudio-silva/cloudflare-fullstack-starter D1 should also have the usernames too ig for name claimed guys, a profile page for them Let's discuss more.

Response: These constraints create a much tighter, faster, and highly focused mobile experience. Dropping synonyms because of the blank-space UI mechanic is exactly the right call—it keeps the cognitive load low and the UI predictable.

Since you are building on the `cloudflare-fullstack-starter` (which typically uses React/Vite, Cloudflare Workers/Hono, and D1), we will design the backend to map cleanly to that stack.

Here is the revised data model, Durable Object state, and API architecture for the backend.

1. D1 Database Schema (Persistent Data)

This handles the word catalog, claimed user profiles, and historical stats for the profile page.

Table: `users`

- `id` (TEXT, Primary Key) - Client UUID.
- `username` (TEXT, Unique) - Claimed name or default generated name.
- `is_claimed` (BOOLEAN) - `true` if they registered an account, `false` for anonymous guests.
- `email` (TEXT, Nullable) - For claimed accounts.
- `total_points` (INTEGER) - Lifetime score for profile page.
- `games_played` (INTEGER) - For calculating all-time averages.

Table: `words`

- `id` (TEXT, Primary Key)
- `word` (TEXT) - The canonical spelling (e.g., "pomegranate"). No synonyms.
- `char_count` (INTEGER) - Driven directly by `word.length`.
- `difficulty` (TEXT) - `easy`, `medium`, `hard`.
- `image_url` (TEXT) - Path to the R2 asset.
- `hint_indices` (TEXT, JSON) - Array of character indexes to reveal. E.g., `[1, 5]` for "broccoli" would reveal `_ r _ _ _ c _ _`. Null for easy/medium words.

Table: `match_history` (Optional, for profile stats)

- `id` (TEXT, Primary Key)
- `user_id` (TEXT, Foreign Key)
- `score` (INTEGER)

- `played_at` (TIMESTAMP)

2. Durable Object State (The Room)

The Durable Object manages the ephemeral, in-memory state of the active room. It handles the WebSockets, the 5-round loop, and the scoring.

In-Memory State Object:

```
{
  roomId: "K9XZ2A",
  status: "LOBBY" | "PLAYING" | "SUMMARY",
  hostId: "user_123", // Reassigned automatically if user_123 disconnects
  currentRound: 1,    // Max 5
  roundEndTime: 1731174600000,
  words: [...],       // Array of 5 word objects fetched from D1 on room creation
  players: {
    "user_123": { username: "SwiftFox84", score: 0, status: "GUESSING", connection: WebSocket },
    "user_456": { username: "Guest99", score: 0, status: "SOLVED", connection: WebSocket }
  }
}
```

Scoring Logic (100 Max): When a correct guess is received: `Score = 50 (Base) + Math.floor(50 * (timeLeft / totalTime))` This keeps points strictly under 100 per round, making a perfect game exactly 500 points.

3. API & WebSocket Architecture

A. Standard HTTP (Hono / tRPC)

- `POST /api/room` - Fetches 5 random words from D1, generates a 6-character room code, and creates the Durable Object stub. Returns the room URL.
- `POST /api/users/claim` - Upgrades an anonymous UUID to a claimed user in D1.
- `GET /api/users/:id` - Fetches user profile stats (total points, average score, games played).

B. WebSocket Events (Client → Durable Object)

- `JOIN_ROOM`: Sends user ID and username.
- `UPDATE_NAME`: Changes the display name in the lobby.
- `START_GAME`: Sent by the `hostId`. If the host drops, the server silently promotes the next oldest connection to host.
- `SUBMIT_GUESS`: Sends the typed string. (e.g., `{ type: "SUBMIT_GUESS", guess: "pomgranate" }`).

C. WebSocket Events (Durable Object → Clients)

- `ROOM_STATE`: Broadcasts the roster, host status, and lobby readiness.
- `ROUND_START`: Payload includes `roundNumber`, `charCount`, `imageUrl`, `timeLimit`, and `hints` (if the word is hard). *Crucially, the actual word is NOT in this payload.*
- `GUESS_FEEDBACK`: Sent only to the guessing player. (e.g., `{ status: "INCORRECT" }` to trigger the mobile shake animation).
- `PLAYER_SOLVED`: Broadcast to all so the UI can show a checkmark next to the player's name without revealing the word.
- `ROUND_END`: Broadcasts the actual word, awards points, and updates the leaderboard.
- `MATCH_SUMMARY`: Final podium payload.

Next Step

Since this is heavily mobile-first and you are using character count blanks (_ _ _ _ _), how do you want to handle the keyboard interaction? Should clicking the blank space just bring up the native iOS/Android keyboard where they type a full string that populates the blanks, or are you envisioning a custom on-screen grid of letters (like Wordle) to prevent mobile autocorrect/autocomplete from ruining the challenge?

User prompt: What else if left to discuss on this now ? :

Response:

User prompt: OS/Android keyboard, but disabled autocorrect/autocomplete pls What else is there to discuss now then ?

Response: Sorry, something went wrong. Please try your request again.
